

RAG & Embeddings

Complete End-to-End Guide — From Scratch to Interview Level

Goal: Understand not only what RAG and embeddings are, but exactly what happens at indexing time and query time, why each component exists, how to implement a basic pipeline, and how to explain it confidently in an interview.

1. The Big Picture

A normal LLM application can look like User → LLM → Answer. The problem appears when the answer must use information that is private, frequently changing, very large, or not reliably present in the model's pretrained knowledge.

Simple LLM application

User question



LLM



Answer

RAG application

User question



Retrieve relevant knowledge



Add retrieved knowledge to the prompt



LLM



Grounded answer

Core idea: RAG does not replace the LLM. It gives the LLM relevant external context at query time.

2. What is RAG?

RAG stands for Retrieval-Augmented Generation. It combines two capabilities: retrieval of relevant information from an external knowledge source and generation of a natural-language answer by an LLM.

- Retrieval: Find information relevant to the user's question.
- Augmentation: Put the retrieved information into the LLM's input/context.
- Generation: Let the LLM generate an answer using that context.

Question



Retrieve relevant chunks



Question + retrieved context



LLM



Answer

3. Why Do We Need RAG?

Suppose a company has an HR handbook containing leave policy, insurance rules, reimbursement rules, and work-from-home policy. A general LLM may not know the company's private and current rules.

company_handbook.pdf

↓

"Employees receive 24 days of annual leave."

User:

"How many annual leaves do I get?"

Sending the entire handbook to the LLM for every question is inefficient. A 500-page document may contain only one paragraph relevant to the question. RAG tries to retrieve that paragraph first.

- Reduces unnecessary context.
- Can use private/internal data.
- Can incorporate changing knowledge without retraining the LLM.
- Can provide source metadata/citations.
- Usually reduces the amount of document text sent to the model.

4. RAG vs Normal LLM vs Fine-Tuning

Approach	What it does	Best fit
Direct LLM	Uses the model's existing knowledge and the current prompt.	General questions and simple tasks.
RAG	Retrieves external information and supplies it as context.	Private, changing, or large knowledge bases.
Fine-tuning	Trains/adapts model parameters using a training dataset.	Changing behavior, style, task patterns, or domain adaptation.

5. End-to-End RAG Architecture

OFFLINE / INDEXING PHASE

Documents
(PDF / DOCX / Web / DB)

↓

Document Loading

↓

Text Parsing

↓

Chunking

↓

Embeddings

↓

Vector Database

↓

Knowledge Base

ONLINE / QUERY PHASE

User Question

↓

Query Embedding

↓

Retriever / Search

↓

Top-K Relevant Chunks

↓

Context Construction

↓

Prompt = Question + Context



LLM



Answer / Citations

This two-phase view is one of the most important interview concepts. Indexing prepares the knowledge base. Query time retrieves from it and generates the answer.

6. Phase A — Indexing the Knowledge Base

6.1 Document Loading

The first step is to read the source data. A document loader converts files or other sources into a programmatically usable document representation.

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("company_policy.pdf")
documents = loader.load()
```

A document commonly contains page content plus metadata such as source filename and page number.

```
Document(
  page_content="Employees receive 24 days of annual leave.",
  metadata={
    "source": "company_policy.pdf",
    "page": 12
  }
)
```

6.2 Text Splitting / Chunking

Large documents are divided into smaller pieces called chunks. Retrieval works at the chunk level because the system normally wants a focused piece of evidence rather than an entire book or PDF.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
splitter = RecursiveCharacterTextSplitter(
  chunk_size=1000,
  chunk_overlap=200
)
```

```
chunks = splitter.split_documents(documents)
```

Chunk size controls approximate chunk length. Chunk overlap repeats some text between neighboring chunks so information near a boundary is less likely to be separated completely.

```
Chunk 1: characters 0 → 1000
Chunk 2: characters 800 → 1800
Chunk 3: characters 1600 → 2600
```

```
Overlap = 200
```

There is no universally perfect chunk size. It depends on document structure, model context, retrieval quality, and the type of questions users ask.

- Too small → context can be fragmented.
- Too large → retrieved chunks contain more irrelevant information.
- Structure-aware chunking can be better for headings, tables, Markdown, code, or long documents.

7. Embeddings — The Core Concept

An embedding is a numerical vector representation of data, such as text, designed so that semantic relationships can be measured in vector space.

"Employees receive 24 days of annual leave."

↓
Embedding model

↓
[0.12, -0.44, 0.73, 0.08, ...]

The exact vector values are not human-readable meanings. The important property is that the representation allows similarity calculations. Semantically related texts tend to have useful proximity in the learned vector space.

"How many vacation days do employees get?"

↓
Query vector

"Employees receive 24 days of annual leave."

↓
Document vector

similarity(query, document)
↓
High similarity

7.1 Embedding Model vs LLM

Component	Input	Output	Main purpose
LLM	Text/messages	Generated text/structured output	Reasoning, generation, transformation
Embedding model	Text or other supported data	Numerical vector	Semantic representation and similarity/search

7.2 What is a Vector?

A vector is simply an ordered list of numbers. An embedding might have hundreds or thousands of dimensions depending on the embedding model.

[0.21, 0.73, -0.14, 0.58, ...]

Do not interpret each dimension as one human concept such as 'dogs' or 'programming'. Modern embeddings are distributed representations; semantic information is encoded across the vector.

7.3 Why Embeddings Enable Semantic Search

Keyword search depends heavily on exact terms. Embedding-based retrieval can match related meaning even when wording differs.

Document:

"Employees are entitled to 24 days of annual leave."

Query:

"How many vacation days do I get?"

Keyword overlap:

vacation ≠ annual leave

Semantic retrieval:

vacation days ≈ annual leave

↓
likely high semantic similarity

8. Similarity Search

Once chunks have been embedded, the system can compare the query vector with stored vectors. Common similarity/distance approaches include cosine similarity, dot product, and Euclidean distance, depending on the model and vector store configuration.

Query vector
↓
Vector database
↓
Compare against stored vectors
↓
Rank by similarity
↓
Return top-K chunks

Top-K means returning the K highest-ranked candidates. If K=5, the retriever attempts to return five relevant chunks rather than all chunks in the database.

9. Vector Databases

A vector database or vector-capable search system stores embeddings together with the information needed to retrieve the associated content. Examples include FAISS, Chroma, Pinecone, Qdrant, Weaviate, and Milvus.

ID: chunk_127
Vector: [0.21, 0.73, ...]
Text: "Employees receive 24 days..."
Metadata:
 source = "hr_policy.pdf"
 page = 12

- Vector: used for similarity search.
- Text/chunk: returned as retrieval context.
- Metadata: source, page, department, date, document type, etc.
- ID: identifies the stored record.

10. Query-Time Retrieval

When the user asks a question, the question is embedded using the same or a compatible embedding setup used to index the documents. The retriever searches the vector store for relevant chunks.

Question:
"How many annual leaves do employees get?"
↓
Embedding model
↓
Query vector
↓
Vector store search
↓
Top-K chunks

11. Context Construction and Generation

Retrieved chunks are assembled into a prompt or message context. The LLM receives both the user question and the retrieved evidence.

You are an assistant.

Use the context below to answer the question.
If the answer is not supported by the context,
say that you do not have enough information.

Context:
Employees receive 24 days of annual leave.
Annual leave requests must be submitted through HR.

Question:
How many annual leaves do employees get?

The LLM can then generate:

"Employees receive 24 days of annual leave."

This is the 'Generation' part of Retrieval-Augmented Generation.

12. Basic RAG Implementation with LangChain

The following example shows the conceptual flow. Exact package names and APIs can change across LangChain versions, so treat this as a learning template and check the current package documentation when implementing a production system.

```
# 1. Load
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("company_policy.pdf")
documents = loader.load()

# 2. Split
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(documents)

# 3. Embeddings
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

# 4. Store
from langchain_chroma import Chroma

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings
)

# 5. Retriever
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 4}
)

# 6. Query
docs = retriever.invoke(
    "How many annual leaves do employees get?"
)
```

At this point, docs contains the retrieved document chunks. The next step is to pass those chunks as context to a chat model.

13. The Complete Execution Trace

INDEXING TIME

PDF
↓
Loader
↓
Documents
↓
Splitter
↓
Chunks
↓
Embedding model
↓
Vectors
↓
Vector database

QUERY TIME

User question
↓
Query embedding
↓
Retriever
↓
Top-K chunks
↓
Context builder
↓
Prompt
↓
LLM
↓
Final answer

14. Metadata and Citations

Metadata makes a RAG system more useful and auditable. Instead of returning only text, the system can retain source, page, document ID, date, department, or other fields.

```
{  
  "source": "hr_policy.pdf",  
  "page": 12,  
  "section": "Annual Leave"  
}
```

A final response can then expose the source:

Employees receive 24 days of annual leave.
Source: hr_policy.pdf, page 12.

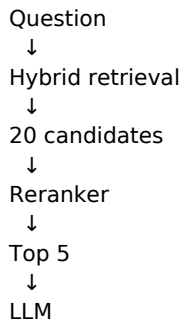
For high-stakes applications, citations should be generated and validated carefully; simply storing metadata does not automatically guarantee that every displayed citation is correct.

15. Retrieval: Beyond Simple Vector Search

A production RAG system may use several retrieval techniques.

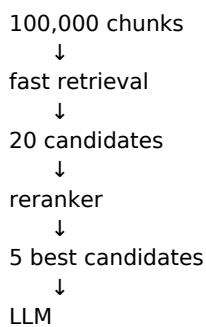
- Dense/vector retrieval: uses embeddings and semantic similarity.
- Keyword/BM25 retrieval: useful for exact names, IDs, codes, and terms.
- Hybrid retrieval: combines semantic and keyword signals.
- Metadata filtering: restricts search to specific sources, dates, departments, access scopes, etc.

- Reranking: takes retrieved candidates and applies a more precise relevance model before sending the final set to the LLM.



16. What is a Reranker?

A retriever is often optimized for quickly finding a broad set of candidates. A reranker can then score those candidates more precisely for the specific query.



This separation can improve relevance while keeping retrieval computationally manageable.

17. Chunking Strategies

- Fixed/character-based: split according to approximate length.
- Recursive splitting: attempt larger natural separators before smaller ones.
- Heading/structure-aware: preserve sections and headings.
- Semantic chunking: use semantic changes to decide boundaries.
- Parent-child retrieval: retrieve small precise chunks but optionally provide a larger parent context.

Interview point: there is no universally optimal chunk size. Chunking should be evaluated using the actual document types and query patterns of the application.

18. Common RAG Failure Modes

18.1 Retrieval Failure

The correct information exists but is not retrieved in the top-K results.

- Improve chunking.
- Improve embedding model choice.
- Use hybrid search.
- Use query rewriting.
- Use metadata filters.
- Add reranking.

18.2 Context Failure

Relevant chunks are retrieved but the final context is too long, redundant, contradictory, or poorly organized.

- Reduce duplicate chunks.
- Rerank and select fewer chunks.
- Compress/summarize retrieved content when appropriate.
- Use clear context formatting.

18.3 Generation Failure

The LLM receives good context but produces an unsupported or incorrect answer.

- Use grounding instructions.
- Ask the model to abstain when evidence is insufficient.
- Require citations where appropriate.
- Evaluate outputs systematically.

18.4 Data/Parsing Failure

The source itself may be poorly extracted, especially PDFs containing tables, scans, images, headers, or complex layouts.

- Use appropriate parsers/OCR.
- Validate extracted text.
- Preserve table and section structure when important.

19. RAG Does Not Automatically Eliminate Hallucinations

This is an important interview nuance. RAG supplies evidence, but the LLM can still misunderstand, ignore, combine, or invent information. A high-quality RAG system therefore evaluates both retrieval quality and answer quality.

Good retrieval + bad generation

↓

Wrong answer

Bad retrieval + good generation

↓

Wrong answer

Good retrieval + grounded generation

↓

Much better chance of a supported answer

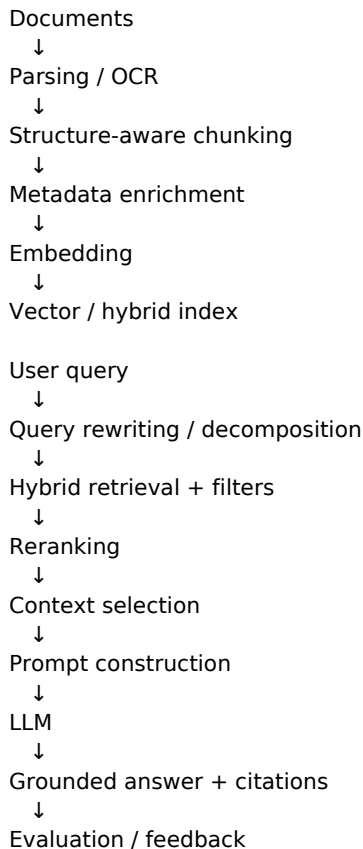
20. RAG Evaluation — Interview Level

Evaluate the system at more than one level.

Level	Question
Retrieval	Did we retrieve the right evidence?
Ranking	Were the most useful chunks near the top?

Context	Did we pass enough relevant evidence without excessive noise?
Generation	Is the answer correct and supported by the retrieved evidence?
Citation	Do citations actually support the claims?
System	Latency, cost, reliability, access control, and scalability.

21. Advanced RAG Pipeline



These are optional production techniques, not requirements for a basic RAG project.

22. Query Rewriting

Sometimes the user's wording is poor for retrieval. A system can first rewrite the query into a clearer search query.

User:
"what about leave?"

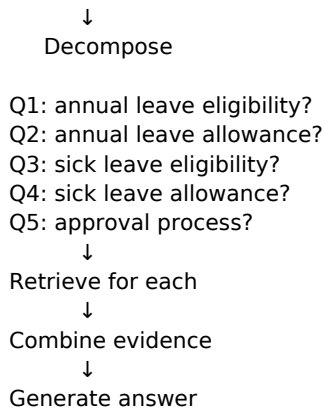
Possible retrieval query:
"employee annual leave policy, eligibility,
annual leave allowance and application process"

Query rewriting can improve retrieval, but it adds model calls, latency, and cost. It should be justified by evaluation.

23. Query Decomposition

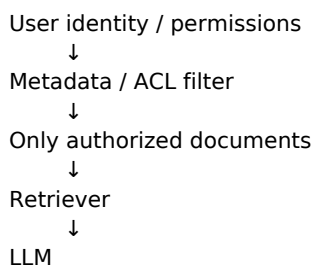
For complex questions, one query may be broken into smaller retrieval questions.

Question:
"Compare annual leave and sick leave eligibility,
allowances, and approval process."



24. Access Control in RAG

A production RAG system must not retrieve documents a user is not allowed to see. Security should be enforced before or during retrieval rather than trusting the LLM to hide restricted information.



Interview point: RAG is not an authorization system. Retrieval must respect the application's access-control rules.

25. RAG vs Long Context

A long-context LLM can accept a large amount of text, but that does not automatically make retrieval unnecessary. Retrieval can reduce irrelevant context, improve cost/latency, and focus the model on the most useful evidence. Conversely, for some smaller or well-bounded document sets, directly supplying the relevant documents may be simpler.

The right architecture depends on document size, query patterns, cost, latency, context limits, and quality requirements.

26. RAG vs Fine-Tuning — Strong Interview Answer

Q: When would you choose RAG instead of fine-tuning?

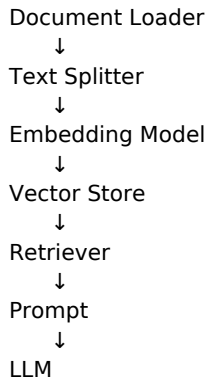
Answer: I would generally choose RAG when the main requirement is giving the model access to external, private, searchable, or frequently changing knowledge. Fine-tuning is more appropriate when I need to change behavior, style, task patterns, or domain adaptation. They can also be combined when a system benefits from both specialized behavior and external retrieval.

Q: Does RAG train the LLM?

Answer: No. Standard RAG does not modify the LLM's parameters. It retrieves external context at inference time and supplies that context to the model.

27. LangChain's Role in RAG

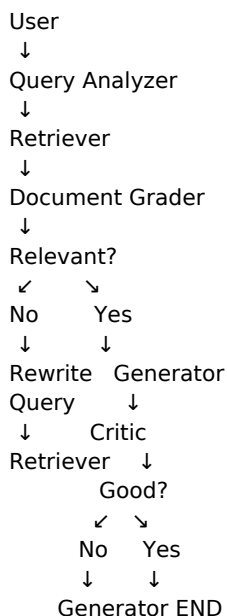
LangChain can provide reusable components for loading documents, splitting text, creating embeddings, connecting vector stores, retrieving documents, building prompts, and calling models.



The exact package/module names vary by integration and LangChain version, so always verify current APIs when writing production code.

28. LangGraph's Role in Advanced RAG

LangGraph is useful when RAG becomes a stateful workflow with decisions, retries, loops, multiple agents, or human approval.



LangChain can supply the retriever, model, tools, and prompts. LangGraph can orchestrate how those pieces move through the workflow.

29. Interview: Explain RAG in 30 Seconds

Model answer: “RAG stands for Retrieval-Augmented Generation. It lets an LLM answer using external knowledge. During indexing, we load documents, split them into chunks, convert the chunks into embeddings, and store those embeddings with the text and metadata in a vector database. At query time, we embed the user's question, retrieve the most relevant chunks using similarity search, add those chunks to the prompt as context, and then ask the LLM to generate a grounded answer.”

30. Interview: Explain Embeddings in 30 Seconds

Model answer: “An embedding is a numerical vector representation of data such as text that captures useful semantic information. We create embeddings for document chunks and store them in a vector index. When a user asks a question, we embed the question using the compatible embedding model and search for nearby or highly similar document vectors. This enables semantic retrieval even when the query and document use different words but have related meanings.”

31. Interview Questions — Fundamentals

Q: What does RAG stand for?

Answer: Retrieval-Augmented Generation.

Q: What are the three words in RAG?

Answer: Retrieval, augmentation, and generation.

Q: What is an embedding?

Answer: A numerical vector representation designed to capture useful semantic relationships.

Q: Why do we chunk documents?

Answer: To create focused retrieval units and avoid treating a very large document as one giant piece of context.

Q: Why use a vector database?

Answer: To efficiently index and retrieve vectors by similarity and associate those vectors with their source text and metadata.

Q: What is Top-K?

Answer: The K highest-ranked retrieval results returned by the search process.

Q: What is a retriever?

Answer: A component that takes a query and returns relevant documents or chunks from a knowledge source.

Q: Does RAG change model weights?

Answer: No, standard RAG does not.

Q: Can RAG use data not seen during model training?

Answer: Yes. That is one of its major purposes: external information can be retrieved at inference time.

Q: Why can semantically similar but differently worded text be retrieved?

Answer: Because the query and document are mapped into a learned vector space where semantic relationships can be compared.

32. Interview Questions — Technical

Q: What is cosine similarity?

Answer: A common similarity measure based on the angle between two vectors. It is often used to compare embedding vectors.

Q: What is hybrid search?

Answer: A retrieval strategy that combines semantic/vector search with lexical or keyword search.

Q: Why might keyword search outperform vector search?

Answer: For exact identifiers, error codes, product IDs, names, or rare strings where exact lexical matching is highly informative.

Q: Why might vector search outperform keyword search?

Answer: When the query and relevant document use different wording but express similar meaning.

Q: What is reranking?

Answer: A second-stage relevance ranking applied to retrieved candidates to choose the most useful evidence for the query.

Q: What happens if the right chunk is not retrieved?

Answer: The LLM may not have the required evidence, so improving retrieval—chunking, embeddings, query processing, hybrid search, filters, or reranking—is necessary.

Q: Can increasing Top-K always improve RAG?

Answer: No. More chunks can increase recall but can also introduce noise, redundancy, latency, cost, and context dilution.

Q: Why is metadata important?

Answer: It supports source attribution, filtering, organization, access control, and traceability.

Q: What is the biggest difference between retrieval and generation?

Answer: Retrieval selects external evidence; generation uses the model to produce the final response.

33. Scenario-Based Interview Questions

Q: Your RAG chatbot gives wrong answers even though the correct information is in the PDF. What would you check first?

Answer: I would separate retrieval from generation. First verify whether the correct chunk is retrieved in the top-K results. If not, inspect parsing, chunking, embeddings, query formulation, filters, hybrid search, and reranking. If the correct chunk is retrieved, inspect context construction and the generation prompt, then evaluate whether the model is grounding its answer in the evidence.

Q: Your system retrieves too much irrelevant information. What can you do?

Answer: Reduce or tune Top-K, improve chunking, add metadata filters, use a reranker, improve query rewriting, use hybrid search, remove duplicate chunks, and evaluate retrieval quality on a representative test set.

Q: Your PDFs contain tables and scanned pages. What problem can occur?

Answer: Text extraction can be incomplete or incorrectly ordered. I would use a parser or OCR approach suitable for the documents and validate the extracted structure before embedding.

Q: The company changes its HR policy every month. RAG or fine-tuning?

Answer: RAG is generally a better fit for the changing knowledge because the knowledge base can be updated without retraining the LLM.

Q: How would you improve a production RAG system?

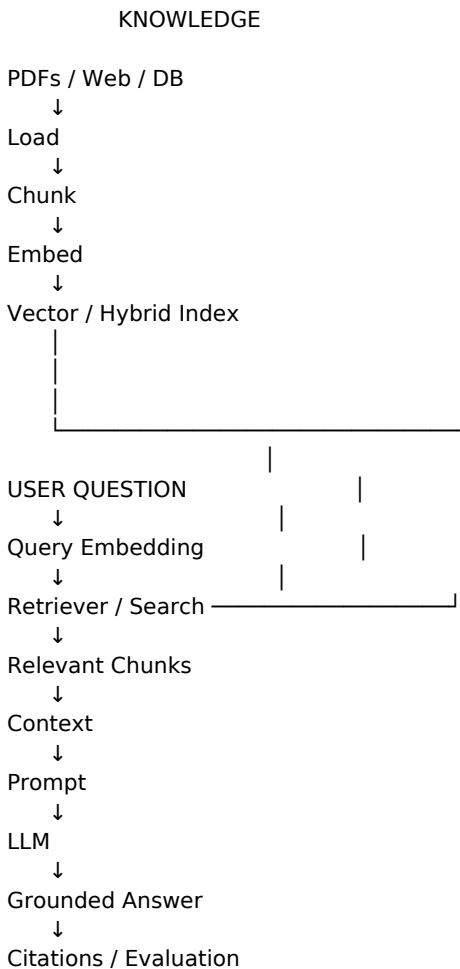
Answer: I would measure retrieval and answer quality separately, improve parsing/chunking, use suitable embeddings, consider hybrid retrieval and reranking, add metadata and access filters, tune Top-K, control context length, add citations, monitor latency/cost, and create a representative evaluation dataset.

34. A Practical Debugging Checklist

- 1. Was the document parsed correctly?
- 2. Are the chunks meaningful and appropriately sized?
- 3. Are important headings/table relationships preserved?
- 4. Are embeddings generated successfully?
- 5. Are query and document embeddings compatible?

- 6. Does the correct chunk appear in Top-K?
- 7. Is the correct metadata/source attached?
- 8. Is the context passed to the LLM correctly?
- 9. Is the prompt asking the model to stay grounded?
- 10. Is the final answer actually supported by the retrieved evidence?
- 11. Are access controls applied before sensitive content reaches the model?
- 12. Are latency and token costs acceptable?

35. Complete Mental Model



36. Final Cheat Sheet

Term	Remember this
RAG	Retrieve external knowledge → add it as context → generate answer.
Embedding	Text/data → numerical vector for semantic comparison.
Chunk	A retrievable piece of a larger document.
Vector DB	Stores/searches vectors and associated data.
Retriever	Finds relevant chunks for a query.
Top-K	Number of highest-ranked candidates returned.

Term	Remember this
Similarity	How close/relevant two vectors are under a chosen metric.
Reranker	Second-stage ranking of retrieved candidates.
Metadata	Source/page/date/ACL/etc. associated with content.
LLM	Uses the question and retrieved context to generate the answer.
LangChain	Provides reusable LLM/RAG application components.
LangGraph	Orchestrates stateful, branching, looping AI workflows.

37. One-Line Answer for the Whole Process

“In RAG, we ingest documents, split them into meaningful chunks, create embeddings for those chunks, store them in a vector or hybrid index, embed the user's query, retrieve and rank the most relevant chunks, construct a context-rich prompt, and let the LLM generate a grounded answer—while evaluating retrieval, generation, citations, security, latency, and cost.”

Study tip: If you can draw the indexing and query-time pipelines from memory and explain why every arrow exists, you have the core RAG interview story.